

Rapport UE3 - Observabilité d'un jeu multijoueur

Victor Chabeau, 08/10/26.

Toutes les heures du rapport sont en UTC, l'export est en heure de Paris (UTC +2) et a été converti en UTC à l'ingestion. Les requêtes logql sur Grafana sont de type Instant, évaluées au 2026-09-27 00:00:01(UTC) sur les 8 jours précédents (on ajoute 1 seconde pour que Loki compte un rapport daté pile à minuit). Les captures d'écran sont dans [docs/captures/](#).

B3 - Qualité des données

Dans Loki j'ai choisi seulement 3 labels: `job` (qui a 2 valeurs), `level` (2 valeurs aussi) et `event` (5 valeurs). Loki et Promtail ajoutent aussi `service_name` (copie de `job`) et `filename`. Loki range les logs dans des groupes à part pour chaque combinaison de labels, donc un label doit avoir le moins de valeurs possible. Je n'ai pas mis en label le client (plusieurs centaines de valeurs), l'id du rapport (environ 2100 valeurs), l'id de la partie (une par partie, 779) ni le build (une nouvelle valeur à chaque version): ils créeraient des milliers de flux et ralentiraient Loki. Ces champs restent dans le contenu des logs, et je les filtre dans les requêtes avec `| json`.

- Volumétrie: l'export couvre bien 7 jours, du 19/09 au 26/09, avec 2889 événements une fois les doublons retirés, dont 779 parties terminées et 2110 spikes (requête 1).
- Format: l'export est un fichier texte brut, pas un JSON par ligne. Mon script `scripts/ingest-history.js` découpe chaque bloc, convertit les dates en UTC et trie les événements avant de les envoyer à Loki. Sans la conversion, toutes les heures seraient décalées de 2 heures. Le format des rapports a aussi changé avec les build 26-6 (le ping passe de `report.rttMs` à `report.network.rttMs`), le script copie la valeur dans un champ `rttMs` commun aux deux versions. 25 rapports sont dans ce nouveau format (requête 3).
- Doublons: 55 événements sont présents deux fois (requête 2). Sans les exclure, les chiffres seraient gonflés d'environ 2%. Mon script ajoute un champ `occurrence` qui numérote les copies d'un même événement (1 pour la première, 2 pour le doublon). Je garde seulement `occurrence = 1` dans toutes mes requêtes.
- Trous: la nuit, il y a peu d'événements, c'est normal car il y a peu de joueurs. Mais le 22/09 de 11:59 à 14:14, aucun événement pendant plus de 2 heures en pleine journée. C'est probablement une panne du serveur ou de la collecte (requête 4). Comme je ne sais pas ce qui s'est passé pendant ces 2 heures, je n'en tire aucune conclusion.
- Incohérences: 70 rapports sont impossibles, ils déclarent plus de 144 images par seconde alors que le jeu est plafonné à 144. Ils viennent tous du même client, `5e1f0c7a` (requête 5). Ces données sont fausses donc je les exclus du calcul de la régression overlay avec `suspect = "false"` (requête 1 de E2).

Les requêtes qui prouvent chaque constat, dans l'ordre (la 4e est en mode Range avec un pas d'1h, pour voir les heures sans événement sur la courbe):

```
sum by (event) (count_over_time({job="telemetry-historique"} | json | occurrence = 1 [8d]))

sum by (event) (count_over_time({job="telemetry-historique"} | json | occurrence > 1 [8d]))

sum by (report_version) (count_over_time({job="telemetry-historique", event="perf_spike"} | json | occurrence = 1 [8d]))

sum(count_over_time({job="telemetry-historique"} | json | occurrence = 1 [1h]))

sum by (client) (count_over_time({job="telemetry-historique", event="perf_spike"} | json | occurrence = 1 | suspect = "true" | regexp `^id:"P-(?P<client>[0-9a-f]+)-` [8d]))
```

C - Métriques

C1 - Métriques exposées

Le service expose ses métriques sur `/metrics` avec prom-client (`src/metrics.js`) et Prometheus les relève toutes les 15 secondes.

- `http_requests_total` (Counter, labels method, route, status): le débit de l'API et la part d'erreurs.
- `http_request_duration_seconds` (Histogram, labels method, route): le temps de réponse de l'API (p95).
- `games_in_progress` (Gauge, sans label): le nombre de parties en cours.
- `games_completed_total` (Counter, labels map, quarantined): les parties terminées par carte, et combien sont en quarantaine.
- `game_duration_seconds` (Histogram, label map): la durée des parties, pour repérer celles qui sont trop courtes (farming).
- `perf_reports_total` (Counter, labels cause, build, source): les rapports de spikes par cause et par build.
- `perf_reports_suspicious_total` (Counter, label build): les rapports impossibles.

Les deux premières métriques couvrent la méthode RED de l'API: le débit (Rate), les erreurs (Errors avec le label status) et la durée (Duration).

- Counter: un nombre qui ne fait que monter utiliser pour compter des événements (requêtes, parties, rapports).
- Histogram: une répartition des valeurs qui permet de calculer le p95 (95% des requêtes sont plus rapides que cette valeur), une moyenne cacherait les requêtes lentes.

- Gauge: une valeur qui monte et descend, ici le nombre de parties en cours à un instant donné.
- Comme pour Loki, chaque valeur de label crée une série dans Prometheus. Je n'ai donc pas mis de label client ni l'URL brute: le label route utilise le modèle de route d'Express, pour qu'une URL inventée ne crée pas une nouvelle série.

C2 - Recording rules

J'ai défini 4 recording rules dans `prometheus/rules/recording.yml`. Prometheus les calcule à l'avance toutes les minutes, ensuite les dashboards et les alertes réutilisent le résultat (capture [docs/captures/C2-recording-rules.png](#)):

- `route:http_requests:rate5m`: nombre de requêtes par seconde par route (dashboard Santé du service).
- `job:http_requests_5xx:ratio_rate5m`: part des réponses en erreur serveur (dashboard Santé du service, alerte ApiErrors).
- `route:http_request_duration_seconds:p95_5m`: temps de réponse p95 par route (dashboard Santé du service, alerte ApiLatencyHigh).
- `build:perf_reports_overlay:ratio_rate5m`: part des spikes overlay par build (dashboard Performance côté joueur, alerte OverlaySpikeRegression).

C3 - Métriques ou logs ?

Dans mon architecture, les deux outils n'ont pas le même rôle:

- Prometheus garde seulement des chiffres: un compteur regroupe des milliers d'événements en une seule valeur, donc il prend peu de place. Il sert à surveiller le service en direct et à déclencher les alertes, pour savoir si tout va bien en ce moment.
- Loki garde le détail de chaque événement (client, écran, navigateur, build...) et tout l'historique importé. C'est lui que j'utilise pour enquêter après coup, quand je cherche qui est touché et pourquoi.
- Les deux se complètent: l'alerte SuspiciousReports de Prometheus prévient qu'un rapport impossible est arrivé, et une requête dans Loki permet de trouver le client responsable (`5e1f0c7a`).
- L'export historique ne peut être que dans Loki: ce sont des événements passés, alors que Prometheus ne fait que relever des chiffres en direct.

E - Détection de comportements

E1 - Typologie

Chaque rapport de l'historique est classé automatiquement à l'ingestion par `src/classify.js` qui ajoute un champ `cause`. Les règles sont testées dans l'ordre de la liste, la première qui correspond donne la cause.

- `network`: le client signale un problème réseau (92 rapports).

- hidden: l'onglet du jeu est passé en arrière-plan (158 rapports).
- shader: compilation de shaders, avec beaucoup de nouveaux programmes graphiques (459 rapports).
- overlay: le dessin de l'overlay dépasse 100ms (262 rapports).
- world: la simulation du monde dépasse 30ms (20 rapports).
- generic: aucune des règles précédentes (1119 rapports).

La signature c'est l'indice dans le rapport qui permet de reconnaître la cause. La requête qui compte les rapports par cause (capture [docs/captures/E1-classification-par-cause.png](#)):

```
sum by (cause) (count_over_time({job="telemetry-historique",
event="perf_spike"} | json | occurrence = 1 [8d]))
```

E2 - Comportements anormaux

- Dégradation liée à une version: la part de spikes overlay passe de 1% à 33% avec le build 24-3. Les premiers spikes overlay sur ce build arrivent dans la soirée du jeudi 24/09, ce qui correspond au jeudi soir dont parle la direction. Les builds suivants n'ont pas corrigé le problème (16% en 26-6, mais sur seulement 25 rapports). Preuves: requête 1, [E2-overlay-par-heure.png](#), [D-performance-du-jeu.png](#).
- Population touchée: presque tous les spikes overlay viennent d'écrans d'au moins 2560px avec le bloom activé (249 sur 262), quel que soit le navigateur. La cause vient donc de la configuration graphique. Preuves: requête 2, [D-performance-du-jeu.png](#).
- Phénomène récurrent: deux problèmes reviennent, mais ils n'ont rien à voir avec la régression overlay. Le serveur saccade certaines nuits vers 01:00, les 21 et 23/09 ([tickGapMaxMs](#) au dessus de 100ms), et la simulation du monde est lente uniquement sur la carte foundry. Preuves: requête 3 et 4, [E2-serveur-saccade.png](#).
- Parties suspectes: une partie normale dure plusieurs minutes. J'ai trouvé 83 parties sur vault de moins de 2 minutes, enchainées sur 2 nuits. Pour moi c'est du farming, des joueurs enchainent des victoires faciles pour gagner des récompenses. Seulement 17 sont en quarantaine, soit environ 1 sur 5, et la quarantaine ne touche que la partie, le joueur lui n'est pas sanctionné. Preuves: requête 5, [E2-farming-vault.png](#), [D-activite-et-integrite.png](#).
- Données falsifiées: le client [5e1f0c7a](#) envoie 70 rapports impossibles, avec plus de 144 images par seconde alors que le jeu est plafonné à 144. Ses rapports sont donc forgés. Preuves: requête 6, [D-activite-et-integrite.png](#).

Les requêtes qui prouvent chaque constat, dans l'ordre (les rapports falsifiés sont exclus de la 1re avec `suspect = "false"`):

```

sum by (report_build) (count_over_time({job="telemetry-historique",
event="perf_spike"} | json | occurrence = 1 | suspect = "false" | cause =
"overlay" [8d]))
/
sum by (report_build) (count_over_time({job="telemetry-historique",
event="perf_spike"} | json | occurrence = 1 | suspect = "false" [8d]))

sum by (report_graphics_width, report_graphics_bloom)
(count_over_time({job="telemetry-historique", event="perf_spike"} | json |
occurrence = 1 | cause = "overlay" [8d]))

sum(count_over_time({job="telemetry-historique", event="perf_spike"} | json
| occurrence = 1 | server_tickGapMaxMs > 100 [8d]))

sum by (server_map) (count_over_time({job="telemetry-historique",
event="perf_spike"} | json | occurrence = 1 | cause = "world" [8d]))

sum by (quarantined) (count_over_time({job="telemetry-historique",
event="game_completed"} | json | occurrence = 1 | map = "vault" | durationS
< 120 [8d]))

sum by (report_build) (count_over_time({job="telemetry-historique",
event="perf_spike"} | json | occurrence = 1 | suspect = "true" [8d]))

```

Les captures sont dans [docs/captures/dashboards/performance-du-jeu/et docs/captures/dashboards/activite-et-integrite/](#).

E3 - Alertes

J'ai transformé ces détections en 5 alertes Prometheus, dans [prometheus/rules/alerts.yml](#). Chaque seuil est placé loin de la valeur normale et proche du problème observé. La durée (**for**) évite d'alerter sur un pic passager.

Un faux positif, c'est une alerte qui sonne sans vrai problème. Un faux négatif c'est un vrai problème qui ne déclenche pas d'alerte.

- **OverlaySpikeRegression**: la part overlay d'un build dépasse 10% pendant 10min. Elle est de 1% en temps normal et de 33% avec la régression. Faux positif: avec peu de trafic, quelques spikes suffisent à dépasser 10%. Faux négatif: si peu de joueurs ont un grand écran avec bloom, la part reste sous 10%.
- **VaultFarming**: plus de 5 parties vault de moins de 120s en 15min, pendant 5min. Une partie normale dure bien plus longtemps. Faux positif: rare, hors farming aucune partie vault ne dure moins de 120s. Faux négatif: un tricheur qui fait des parties plus longues, farme lentement ou change de carte.
- **SuspiciousReports**: au moins 1 rapport impossible en 10min, car un client honnête n'en envoie jamais. Faux positif: rare, aucun client honnête dans l'historique. Faux négatif: un tricheur qui garde des valeurs crédibles (sous 144fps).

- ApiLatencyHigh: le p95 de `/api/reports` dépasse 200ms pendant 5min, alors qu'il est d'environ 10ms en temps normal. Faux positif: un pic court quand il y a peu de requêtes. Faux négatif: si seulement quelques requêtes sont lentes, le p95 ne bouge pas.
- ApiErrors: plus de 5% de réponses 5xx pendant 5min, alors qu'il y en a 0% en temps normal. Faux positif: avec peu de trafic, 1 erreur suffit à dépasser 5%. Faux négatif: si le service est arrêté, il n'y a plus de requêtes donc pas d'alerte.

Sur la capture `docs/captures/E3-alertes.png`, OverlaySpikeRegression et VaultFarming sont en Firing, donc les alertes se déclenchent bien sur les vrais problèmes.

Limite: aucune alerte ne prévient si le service est complètement arrêté. il faudrait en ajouter une sur `up == 0`.

F - Postmortem

Réponses aux questions de la direction

- Pourquoi des saccades depuis jeudi soir? Le build 24-3, mis en ligne le jeudi 24/09, contient une régression: le dessin de l'overlay devient très lent sur certaines configurations.
- Qui est touché? Les joueurs qui ont une largeur d'écran d'au moins 2560 px avec le bloom activé, quel que soit leur navigateur. Pendant un spike, l'image se fige et le jeu devient injouable.
- Est-ce que quelqu'un abuse du jeu? Oui, de deux façons: le client `5e1f0c7a` envoie de faux rapports de performance, et des joueurs ont fait du farming sur la carte vault pendant deux nuits.

Chronologie

Les heures sont en UTC, avec l'heure de Paris entre parenthèses.

- Nuit du 22 au 23/09, de 23:00 à 02:00 environ (01:00 à 04:00): farming sur vault, nuit 1.
- Jeudi 24/09: mise en ligne du build 24-3.
- Soirée du jeudi 24/09: premiers spikes overlay sur le build 24-3.
- Nuit du 24 au 25/09, de 22:00 à 01:00 environ (00:00 à 03:00): farming sur vault, nuit 2.
- 26/09: builds 26-5 et 26-6, toujours touchés par la régression.

Cause

Avec le build 24-3, le dessin de l'overlay (viseur, barre de vie, mini carte) dépasse 100ms par image. Cela arrive presque uniquement avec un grand écran et le bloom activé. La part des spikes overlay passe de 1% avant ce build à 33% après (voir E2).

Preuves

- Requêtes: les requêtes 1 et 2 de E2 pour la régression et la population touchée, les requêtes 5 et 6 pour le farming et la falsification.

- Captures: [docs/captures/dashboards/performance-du-jeu/E2-overlay-par-heure.png](#) (la hausse commence jeudi soir), [docs/captures/dashboards/performance-du-jeu/D-performance-du-jeu.png](#) (builds, écrans, navigateurs), [docs/captures/dashboards/activite-et-integrite/D-activite-et-integrite.png](#) (client falsifié, farming), [docs/captures/E3-alertes.png](#) (alertes déclenchées).

Actions correctives

- Revenir au build 22-1, ou désactiver le bloom sur les grands écrans en attendant un correctif.
- Corriger le dessin de l'overlay dans un nouveau build.
- Bloquer le client [5e1f0c7a](#) et ignorer ses rapports.
- Mettre en quarantaine toutes les parties vault de moins de 120s, et sanctionner le joueur, pas seulement la partie.

Actions préventives

- Tester les performances en 2560px et en 4k avec le bloom actif avant chaque nouveau build.
- Déployer un nouveau build d'abord sur une partie de joueurs, pour voir un problème avant qu'il touche tout le monde.
- Vérifier les rapports côté serveur et refuser ceux qui sont impossibles (plus de 144fps, partie inconnue).
- Garder les alertes de la partie E3, et en ajouter une qui prévient si le service est arrêté ([up == 0](#)).